

## Correcciones

### Revisión Modelo de Datos — Fase 1 MVP

#### Observaciones y soluciones para el equipo

##### 1. Firma digital — cobertura parcial

###### Observación

clinical\_documents registra el resultado de la firma del médico (signed\_by, signed\_at, signature\_url), pero la arquitectura §4.3.1 especifica campos de firma digital tanto para el médico como para el paciente. No existe estructura para capturar la firma del paciente ni para distinguir múltiples firmantes por documento. Además, si en el futuro se requiere auditar quién firmó qué y con qué método, tres columnas sueltas en la tabla del documento no son suficientes.

###### Solución

Crear una tabla clinical.document\_signatures que normalice todas las firmas de un documento, independientemente del rol del firmante:

sql

```
CREATE TABLE clinical.document_signatures (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  clinical_document_id UUID NOT NULL REFERENCES clinical.clinical_documents(id),  
  signer_user_id  UUID NULL REFERENCES core.users(id),    -- NULL si el firmante es  
  el paciente  
  signer_patient_id  UUID NULL REFERENCES patients.patients(id), -- NULL si el  
  firmante es staff  
  signer_role      VARCHAR(40) NOT NULL,                  -- 'doctor' | 'patient' | 'witness'  
  signature_url    TEXT NOT NULL,                        -- URL imagen de firma en object  
  storage  
  signature_method VARCHAR(30) NOT NULL,                  -- 'drawn' | 'typed' |  
  'biometric'  
  ip_address      INET NULL,  
  signed_at       TIMESTAMPTZ NOT NULL DEFAULT now(),  
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

```
CREATE INDEX ON clinical.document_signatures (clinical_document_id);  
CREATE INDEX ON clinical.document_signatures (signer_patient_id);
```

Con esto, clinical\_documents conserva signed\_at como la fecha del primer cierre del documento (útil para ordenar), pero el detalle de quién firmó, cómo y cuándo queda en esta tabla. Se eliminan las columnas signed\_by, signed\_at y signature\_url de clinical\_documents o se marcan como deprecated y se dejan de escribir en favor de esta tabla, según la estrategia de migración que elija el equipo.

## 2. Ventana de conciliación post-cita — cobertura parcial

### Observación

Esta es la brecha más crítica del modelo porque afecta directamente la integridad del Momento 2 del ARE (asiento de costo de insumos). El flujo descrito en §4.4.2 es: el médico confirma las cantidades reales consumidas en una ventana post-cita → se publica el evento ConsumoConciliado → el ARE genera la póliza con Débito 51000 / Crédito 12100.

El problema es que no existe ninguna tabla donde se persistan esas cantidades reales por ítem. appointment\_procedures solo registra qué procedimientos se ejecutaron en la cita, pero no las cantidades reales de cada insumo. Sin esa tabla, el evento ConsumoConciliado no tiene de dónde leer los datos concretos para el asiento, y el ARE tendría que usar las cantidades presupuestadas de procedure\_supplies, lo cual rompe el principio de exactitud contable que establece §4.7.2: "usando exactamente las cantidades validadas en la ventana de conciliación".

### Solución

Agregar una tabla clinical.conciliation\_items que registre las cantidades reales validadas por el médico o asistente para cada insumo, vinculada a la cita y al procedimiento ejecutado:

sql

```
CREATE TABLE clinical.conciliation_items (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  appointment_id  UUID NOT NULL REFERENCES scheduling.appointments(id),  
  appointment_procedure_id  UUID NOT NULL REFERENCES  
scheduling.appointment_procedures(id),  
  product_id     UUID NOT NULL REFERENCES inventory.products(id),  
  quantity_budged  NUMERIC(10,4) NOT NULL, -- snapshot de  
procedure_supplies.quantity_required al momento de la cita  
  quantity_actual  NUMERIC(10,4) NOT NULL, -- cantidad real validada en la  
ventana de conciliación  
  unit           VARCHAR(20) NOT NULL,
```

```

    unit_cost_snapshot    NUMERIC(12,4) NOT NULL, -- costo unitario del producto al
momento del consumo
    total_cost            NUMERIC(14,4) GENERATED ALWAYS AS (quantity_actual *
unit_cost_snapshot) STORED,
    notes                 TEXT NULL,
    reconciled_by         UUID NOT NULL REFERENCES core.users(id),
    reconciled_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
    created_at            TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

```

CREATE INDEX ON clinical.conciliation_items (appointment_id);
CREATE INDEX ON clinical.conciliation_items (product_id);
CREATE INDEX ON clinical.conciliation_items (appointment_procedure_id);

```

quantity\_budged y unit\_cost\_snapshot son snapshots tomados al momento de la conciliación, no FKs dinámicas, porque los precios y cantidades del catálogo pueden cambiar y el registro contable debe ser inmutable en su contexto histórico.

El payload del evento ConsumoConciliado que publica el módulo clínico hacia el ARE debe incluir el array de conciliation\_items con sus total\_cost, de modo que el ARE no necesite volver a calcular nada: solo transforma cada ítem en líneas de asiento Débito 51000 / Crédito 12100.

### 3. Validación de partida doble — cobertura parcial

#### Observación

La columna is\_balanced en journal\_entries registra si la póliza cumple la condición  $\Sigma$  débitos –  $\Sigma$  créditos = 0, pero el modelo de datos no especifica el mecanismo que ejecuta esa validación antes de persistir. §2.6 y §4.8 de la arquitectura establecen que si la condición falla, la transacción completa debe hacer ROLLBACK y registrar el intento fallido. Si esa validación vive solo en la capa de aplicación, un bug o un insert manual directo sobre la base de datos puede generar pólizas desbalanceadas sin que nadie lo detecte.

#### Solución

Implementar la validación en dos capas:

Capa 1 — Trigger en PostgreSQL (defensa de última línea, no salteable desde aplicación):

```
sql
```

```

CREATE OR REPLACE FUNCTION accounting.validate_journal_balance()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_sum_debit NUMERIC;
    v_sum_credit NUMERIC;
BEGIN
    SELECT
        COALESCE(SUM(debit), 0),
        COALESCE(SUM(credit), 0)
    INTO v_sum_debit, v_sum_credit
    FROM accounting.journal_lines
    WHERE journal_entry_id = NEW.journal_entry_id;

    IF v_sum_debit <> v_sum_credit THEN
        RAISE EXCEPTION 'Partida doble inválida: débitos=% créditos=% en póliza %',
            v_sum_debit, v_sum_credit, NEW.journal_entry_id;
    END IF;

    -- Actualiza is_balanced en la cabecera
    UPDATE accounting.journal_entries
    SET is_balanced = TRUE
    WHERE id = NEW.journal_entry_id;

    RETURN NEW;
END;
$$;

```

```

-- Se dispara AFTER INSERT en journal_lines, no BEFORE,
-- para que pueda leer el conjunto completo de líneas insertadas en la transacción.
CREATE CONSTRAINT TRIGGER trg_validate_journal_balance
AFTER INSERT ON accounting.journal_lines
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW EXECUTE FUNCTION accounting.validate_journal_balance();

```

Se usa DEFERRABLE INITIALLY DEFERRED para que el trigger se ejecute al momento del COMMIT, no después de cada INSERT individual de línea, permitiendo insertar débito y crédito dentro de la misma transacción sin que el trigger falle en el estado intermedio.

Capa 2 — Registro del intento fallido en domain\_events (desde la capa de aplicación, en el catch del ROLLBACK):

Cuando el trigger lanza la excepción y la transacción hace ROLLBACK, el ARE debe capturar el error y escribir un registro en `accounting.domain_events` con:

```
event_type = 'JournalBalanceError'
```

```
processed_at = NULL (queda como evento fallido)
```

```
processing_error = <mensaje de la excepción>
```

```
payload = <snapshot del intento de póliza>
```

Esto garantiza trazabilidad del fallo sin que quede en ningún log de aplicación que pueda rotarse.

#### 4. Contenido del ticket de venta — brecha

##### Observación

`cash.payments` tiene `ticket_number` como identificador único del ticket, pero no almacena el contenido del mismo. Un ticket de venta necesita persistir sus líneas de detalle (qué procedimientos o conceptos se cobraron, con sus importes individuales), la forma de pago desglosada, y el total. Esto es necesario para reimpresión, para el historial del paciente y para el cuadre de caja. En Fase 1 no hay CFDI, así que el ticket es el único comprobante que recibe el paciente.

##### Solución

Dos opciones; se recomienda la primera para mantener consistencia con el patrón JSONB ya usado en el modelo:

Opción A — Columna JSONB en `payments` (recomendada para Fase 1):

Agregar a `cash.payments`:

```
sql
```

```
ALTER TABLE cash.payments
```

```
ADD COLUMN ticket_lines JSONB NOT NULL DEFAULT '[]';
```

Estructura del JSON:

```
json
```

```
{
  "lines": [
    {
      "description": "Consulta general",
      "procedure_id": "uuid-opcional",
```

```

    "quantity": 1,
    "unit_price": 800.00,
    "discount_pct": 0,
    "subtotal": 800.00
  },
  {
    "description": "Limpieza dental",
    "procedure_id": "uuid-opcional",
    "quantity": 1,
    "unit_price": 600.00,
    "discount_pct": 10,
    "subtotal": 540.00
  }
],
"subtotal": 1340.00,
"total": 1340.00,
"payment_breakdown": [
  { "method": "cash", "amount": 1000.00 },
  { "method": "card", "amount": 340.00 }
]
}

```

Agregar índice GIN para búsquedas por concepto si se requiere en el futuro:

sql

```
CREATE INDEX ON cash.payments USING GIN (ticket_lines);
```

Opción B — Tabla normalizada cash.payment\_lines (recomendada si en Fase 2 se necesita reportería por concepto cobrado):

sql

```

CREATE TABLE cash.payment_lines (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  payment_id  UUID NOT NULL REFERENCES cash.payments(id),
  clinic_id   UUID NOT NULL REFERENCES core.clinics(id),
  procedure_id UUID NULL REFERENCES treatments.procedures(id), -- NULL si es
concepto libre
  description TEXT NOT NULL,
  quantity    NUMERIC(8,2) NOT NULL DEFAULT 1,
  unit_price  NUMERIC(12,2) NOT NULL,
  discount_pct NUMERIC(5,2) NOT NULL DEFAULT 0,
  subtotal    NUMERIC(12,2) NOT NULL,

```

```
sort_order INT NOT NULL DEFAULT 0,  
created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

```
CREATE INDEX ON cash.payment_lines (payment_id);  
CREATE INDEX ON cash.payment_lines (clinic_id, procedure_id);
```

Para Fase 1, la Opción A es más rápida de implementar y suficiente. La Opción B se justifica si desde el inicio se quiere reportar ingresos desglosados por procedimiento desde la caja, sin depender del Libro Mayor.

## 5. Alertas de stock mínimo y caducidad — brecha

### Observación

inventory.products tiene correctamente stock\_min, stock\_max, has\_expiry y expiry\_alert\_days como parámetros de configuración de alertas. Sin embargo, no existe ninguna tabla que registre las alertas generadas, su estado (leída, resuelta, ignorada) ni a quién se enviaron. Sin esto, cada vez que el sistema evalúe las condiciones de alerta, no sabe si ya notificó sobre ese producto o si el administrador ya lo vio, generando potencialmente alertas duplicadas en cada ciclo de evaluación.

### Solución

Crear un schema o tabla de notificaciones/alertas. Se recomienda una tabla genérica que sirva también para otros tipos de alerta futuros (caducidad de sesión, diferencias de caja, etc.):

sql

```
CREATE TABLE core.system_alerts (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  clinic_id   UUID NOT NULL REFERENCES core.clinics(id),  
  alert_type  VARCHAR(60) NOT NULL,  
  -- 'stock_below_min' | 'stock_above_max' | 'expiry_warning' | 'expiry_expired' |  
  'cash_difference'  
  severity    VARCHAR(20) NOT NULL DEFAULT 'warning',  
  -- 'info' | 'warning' | 'critical'  
  entity_type VARCHAR(60) NOT NULL,  
  -- 'product' | 'cash_session' | etc.  
  entity_id   UUID NOT NULL,  
  -- ID del producto, sesión de caja, etc.  
  title       TEXT NOT NULL,
```

-- Texto en lenguaje natural, ej: "Clorhexidina 0.12% por debajo del mínimo (3 unidades restantes)"

```
metadata    JSONB NULL,
-- Datos adicionales: { current_stock: 3, stock_min: 10, product_name: "..."}
status      VARCHAR(20) NOT NULL DEFAULT 'pending',
-- 'pending' | 'read' | 'resolved' | 'snoozed'
read_by     UUID NULL REFERENCES core.users(id),
read_at     TIMESTAMPTZ NULL,
resolved_by  UUID NULL REFERENCES core.users(id),
resolved_at  TIMESTAMPTZ NULL,
snoozed_until TIMESTAMPTZ NULL,
created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

```
CREATE INDEX ON core.system_alerts (clinic_id, status, created_at DESC);
CREATE INDEX ON core.system_alerts (clinic_id, alert_type, entity_id);
CREATE INDEX ON core.system_alerts (entity_id);
```

El índice compuesto (clinic\_id, alert\_type, entity\_id) permite al proceso de evaluación verificar eficientemente si ya existe una alerta activa (status = 'pending') para un producto dado, evitando duplicados:

sql

-- Antes de insertar una alerta, verificar si ya existe una activa para el mismo producto:

```
SELECT id FROM core.system_alerts
WHERE clinic_id = $1
  AND alert_type = 'stock_below_min'
  AND entity_id = $2 -- product_id
  AND status = 'pending'
LIMIT 1;
```

-- Si existe, no insertar. Si no existe, insertar.

Para caducidades, el proceso de evaluación debería correr diariamente (cron job o pg\_cron) y consultar:

sql

```
SELECT p.id, p.name, sm.expiry_date, sm.lot_number, sm.stock_after
FROM inventory.products p
JOIN inventory.stock_movements sm ON sm.product_id = p.id
WHERE p.clinic_id = $1
  AND p.has_expiry = TRUE
  AND sm.expiry_date IS NOT NULL
```



```
AND sm.expiry_date <= CURRENT_DATE + (p.expiry_alert_days || ' days')::INTERVAL
AND sm.movement_type = 'entry'
ORDER BY sm.expiry_date ASC;
```